

Mini Assessment

API Error Handling and Safe JSON Access

Name: BHAJINE ROHITH SAI NAIK

Reg. No.: 99230040083

Department: CSE – AIML

1. If an API returns 429, what should your program do?

Answer:

A 429 response means Too Many Requests, usually because the API rate limit has been exceeded. The program should temporarily stop sending requests and wait before trying again. If the server provides a Retry-After value, it should be respected. Otherwise, use exponential backoff. After a reasonable number of retries, the program should stop and handle the failure gracefully.

- HTTP 429 means "Too Many Requests" – it is a rate-limit signal, not a server crash.
- Always check for a Retry-After header first before using a fixed or exponential wait time.
- Exponential backoff increases wait time between retries (e.g., 1s, 2s, 4s, 8s) to avoid overwhelming the server.
- Set a maximum retry limit so the program does not retry forever.

2. If response.json() fails, what should your program do?

Answer:

The program should catch the JSON parsing error so that the application does not crash. It should check the HTTP status and, when useful, inspect the response text for logging or debugging. The error should be handled with a clear message, and the program should either continue safely or terminate the operation in a controlled way.

- Wrap response.json() in a try/except block to prevent the program from crashing.
- Check response.status_code before parsing, since a non-2xx status often explains why JSON parsing failed.
- Log the raw response text when parsing fails – it helps with debugging malformed or empty responses.
- Decide whether to retry, skip, or stop the operation once the error is caught, rather than letting it fail silently.

3. If a JSON key is missing, why might .get() be safer than []?

Answer:

data.get("key") is safer when a key may be missing because it returns None, or another specified default value, instead of raising an exception. In contrast, data["key"] expects the key to exist and raises a KeyError when it is absent. Therefore, .get() is useful for optional or uncertain JSON fields.

- data.get("key") returns None (or a chosen default) when the key is absent, instead of raising an error.
- data["key"] raises a KeyError if the key does not exist, which can crash the program if unhandled.
- Use .get("key", default_value) to supply a fallback value for optional fields.
- Use ["key"] only when the field is guaranteed to always be present in the response.